

Experiment-04

Aim:

- A. You are given a table `tbl_happiness` that contains the ranking of happiness for different countries. Your task is to retrieve all the records from the table while ensuring that "India" and "Sri Lanka" appear at the top, followed by the rest of the countries in their original ranking order. Make all the rankings in descending order except India and Sri Lanka.
- B. A company maintains employee information in a single table where each employee may report to another employee acting as their manager. Prepare a report that displays the details of every employee who supervises at least one other employee. The report should include the manager's employee ID, name, total number of direct reports, and the average age of those reporting employees. The average age should be rounded to the nearest whole number and the final result should be sorted by employee ID.

Tools / Platform Used:

- Windows
- PostgreSQL
- PgAdmin

Theory:

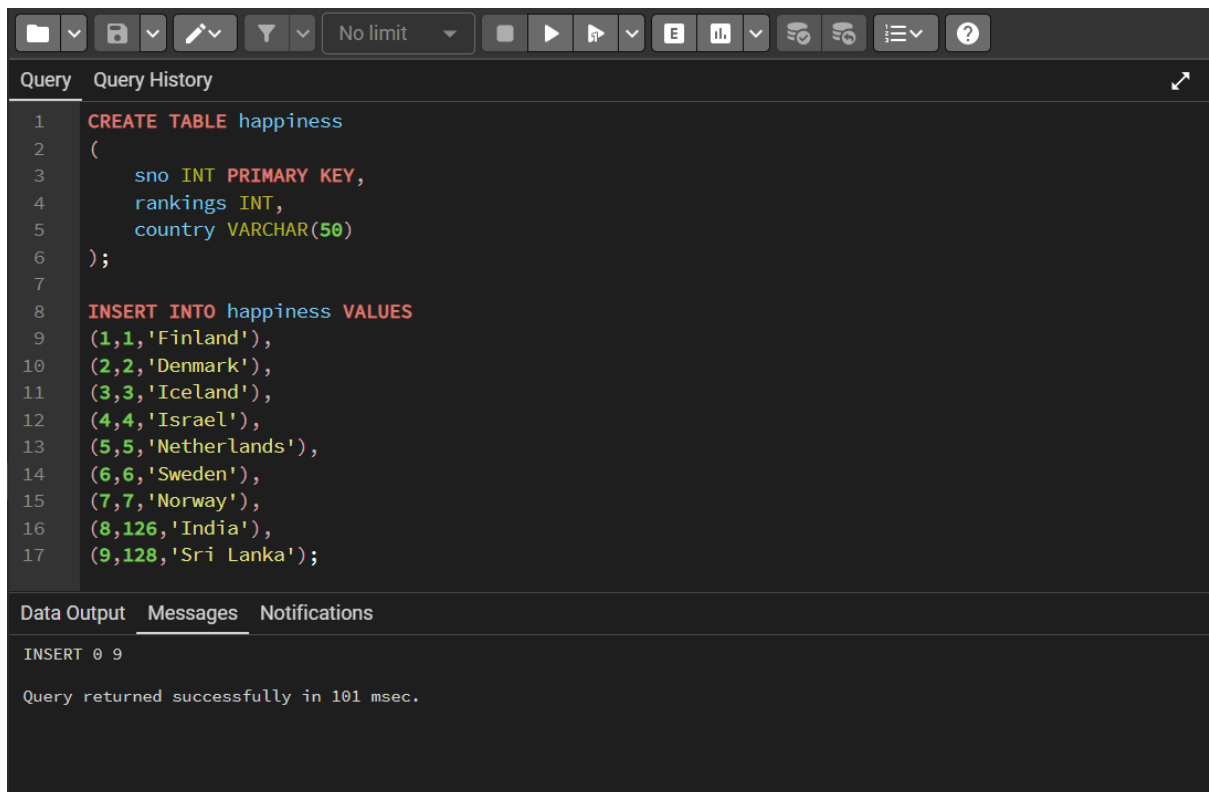
The CASE expression is a conditional expression used to return different values depending on one or more conditions. It is commonly used in:

- SELECT statements
- UPDATE statements
- ORDER BY
- WHERE clauses

It is similar to an if-else-if ladder or a switch-case statement in programming languages.

Code and Output:

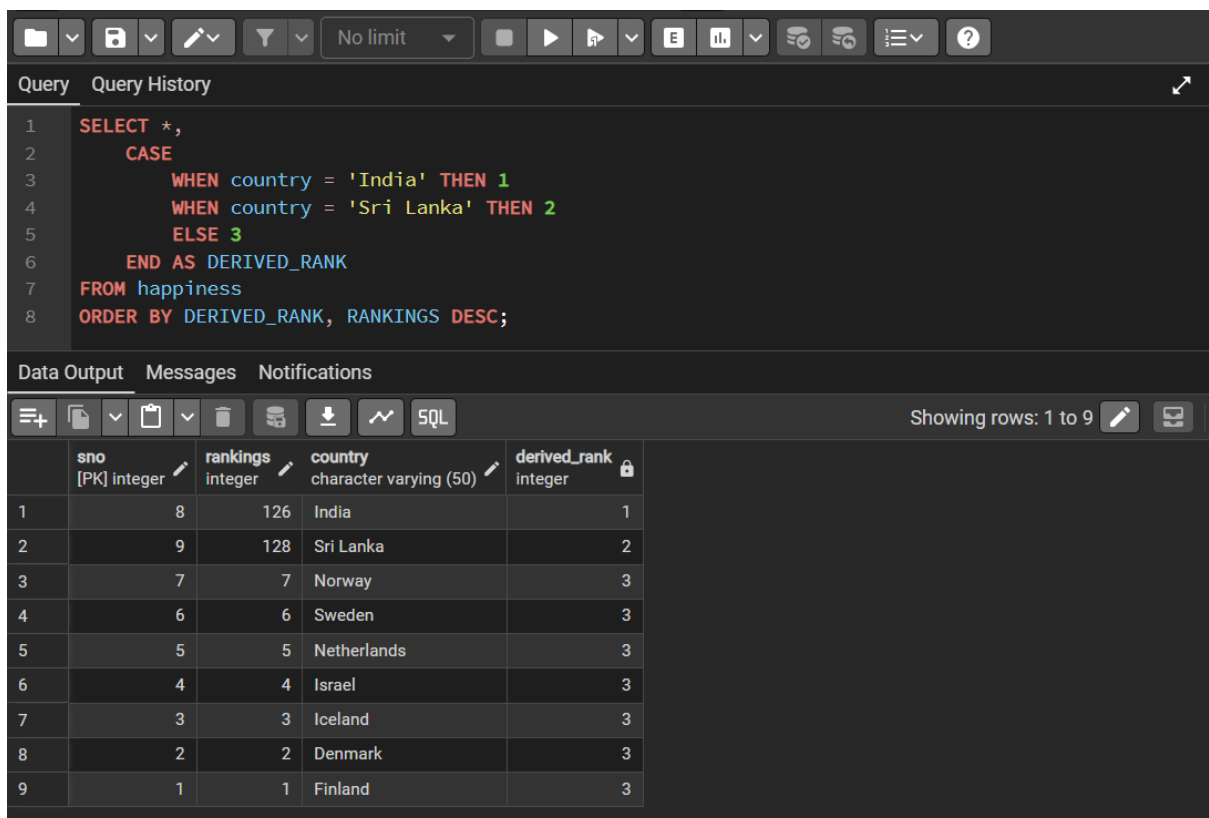
A.



The screenshot shows a SQL IDE interface. The top toolbar includes icons for file operations, query execution, and help. The main editor displays the following SQL code:

```
1 CREATE TABLE happiness
2 (
3     sno INT PRIMARY KEY,
4     rankings INT,
5     country VARCHAR(50)
6 );
7
8 INSERT INTO happiness VALUES
9 (1,1,'Finland'),
10 (2,2,'Denmark'),
11 (3,3,'Iceland'),
12 (4,4,'Israel'),
13 (5,5,'Netherlands'),
14 (6,6,'Sweden'),
15 (7,7,'Norway'),
16 (8,126,'India'),
17 (9,128,'Sri Lanka');
```

Below the editor, the 'Data Output' tab is active, showing the message: 'INSERT 0 9' and 'Query returned successfully in 101 msec.'



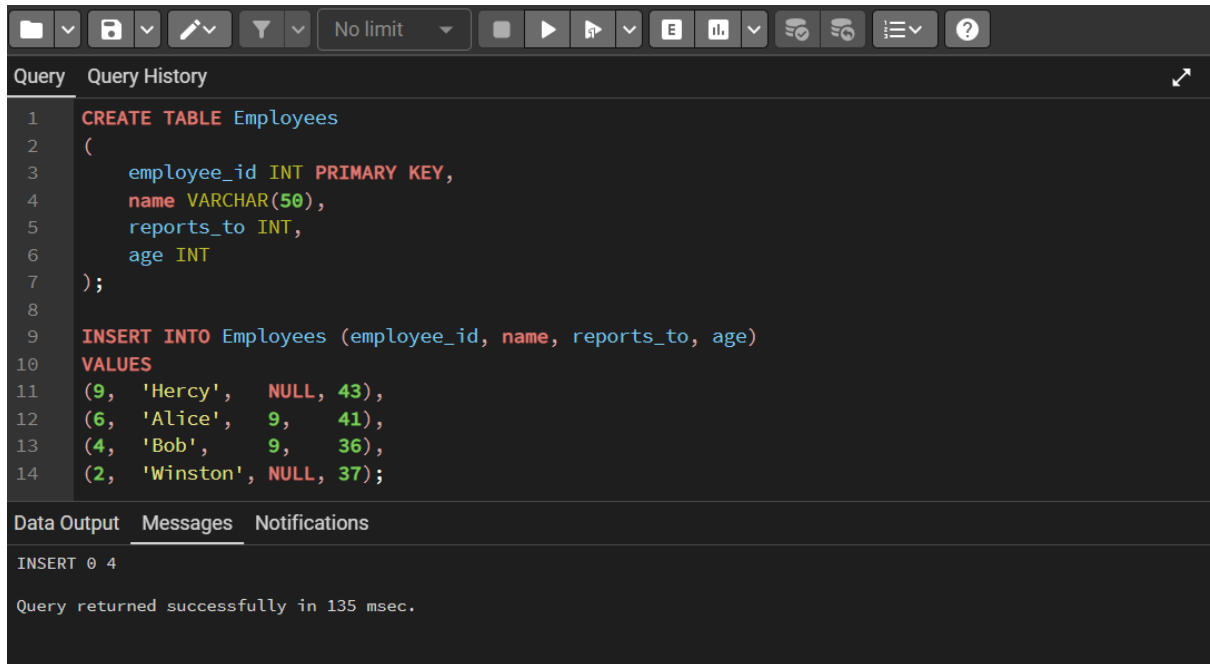
The screenshot shows the same SQL IDE interface. The main editor displays the following SQL query:

```
1 SELECT *,
2     CASE
3         WHEN country = 'India' THEN 1
4         WHEN country = 'Sri Lanka' THEN 2
5         ELSE 3
6     END AS DERIVED_RANK
7 FROM happiness
8 ORDER BY DERIVED_RANK, RANKINGS DESC;
```

Below the editor, the 'Data Output' tab is active, showing a table with 9 rows. The table has columns: sno, rankings, country, and derived_rank. The data is sorted by derived_rank and then by rankings in descending order.

	sno [PK] integer	rankings integer	country character varying (50)	derived_rank integer
1	8	126	India	1
2	9	128	Sri Lanka	2
3	7	7	Norway	3
4	6	6	Sweden	3
5	5	5	Netherlands	3
6	4	4	Israel	3
7	3	3	Iceland	3
8	2	2	Denmark	3
9	1	1	Finland	3

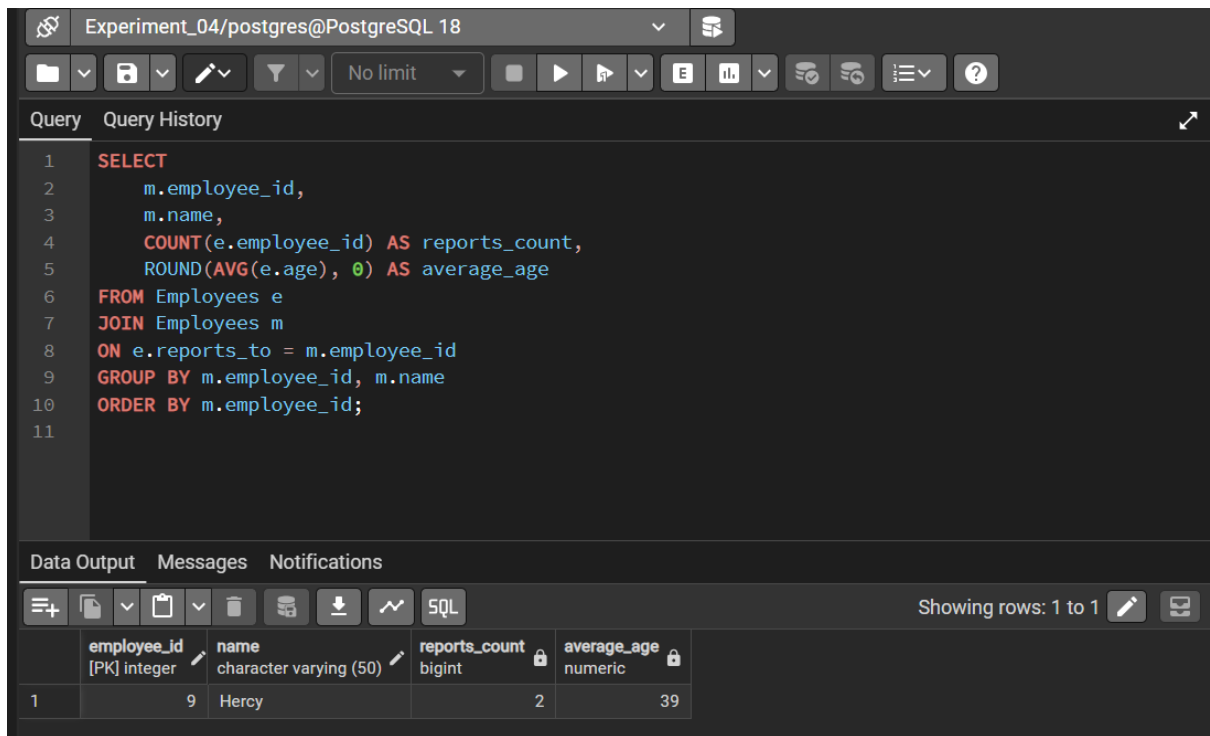
B.



The screenshot shows a SQL IDE interface with a dark theme. The top toolbar includes icons for file operations, query execution, and settings. The main query editor contains the following SQL code:

```
1 CREATE TABLE Employees
2 (
3     employee_id INT PRIMARY KEY,
4     name VARCHAR(50),
5     reports_to INT,
6     age INT
7 );
8
9 INSERT INTO Employees (employee_id, name, reports_to, age)
10 VALUES
11 (9, 'Hercy', NULL, 43),
12 (6, 'Alice', 9, 41),
13 (4, 'Bob', 9, 36),
14 (2, 'Winston', NULL, 37);
```

Below the query editor, the 'Data Output' tab is active, displaying the message: 'INSERT 0 4' and 'Query returned successfully in 135 msec.'



The screenshot shows the same SQL IDE interface. The query editor contains the following SQL code:

```
1 SELECT
2     m.employee_id,
3     m.name,
4     COUNT(e.employee_id) AS reports_count,
5     ROUND(AVG(e.age), 0) AS average_age
6 FROM Employees e
7 JOIN Employees m
8 ON e.reports_to = m.employee_id
9 GROUP BY m.employee_id, m.name
10 ORDER BY m.employee_id;
```

The 'Data Output' tab is active, showing a table with 4 columns: employee_id, name, reports_count, and average_age. The table contains one row of data for employee_id 9 (Hercy).

	employee_id [PK] integer	name character varying (50)	reports_count bigint	average_age numeric
1	9	Hercy	2	39

Result:

The queries for the given tasks were written and executed successfully. The tasks given were completed successfully.

Learning Outcomes:

- Practiced PostgreSQL
- Got hands-on experience on working with Switch-Case Statements in PostgreSQL.
- Practiced commands of Switch-Case in SQL.